Monolithic Architecture

systemdesignschool.io/fundamentals/microservices

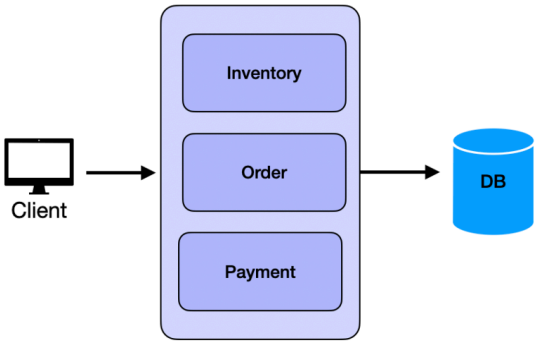
Microservices and Monolithic Architecture

What It Is

A monolithic architecture is a single unified application where all components are tightly coupled and run as a single process. The entire application, from the user interface to business logic and data access, resides in one codebase and is deployed together.

Example: E-Commerce Store

In a monolithic architecture, the e-commerce store would look something like this:



All functionalities—from managing products to processing payments—are part of a single application.

Pros

- Simplicity: Easy to develop, test, and deploy when the application is small.
- Performance: Tight coupling often results in faster communication between components.
- Ease of Debugging: All the code is in one place, making it easier to trace issues.
- Lower Initial Cost: Less overhead in terms of managing infrastructure and development processes.

Cons

- Reliability: A failure in one part of the application can bring down the entire system.
- Scalability: Difficult to scale specific parts of the application independently.
- Complexity at Scale: As the application grows, the codebase becomes harder to maintain. For example, a new developer on the payment team needs to learn how things work in inventory and orders modules.
- Deployment Bottlenecks: Changes to one module require redeploying the entire application.

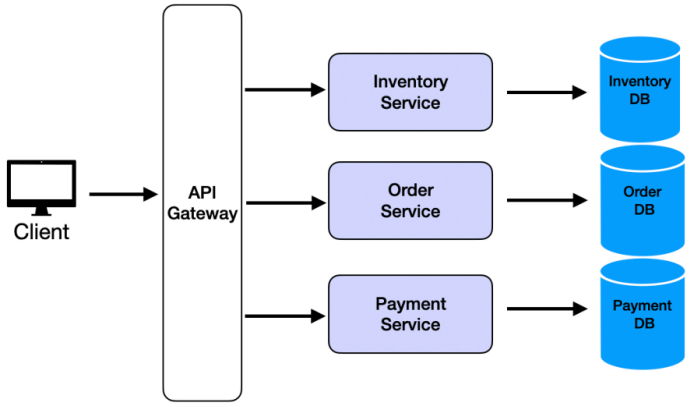
Microservices Architecture

What It Is

Microservices architecture breaks down an application into smaller, loosely coupled services, each responsible for a specific functionality with each service having its own database. These services communicate with each other over APIs.

Example: E-Commerce Store

In a microservices architecture, the e-commerce store would look like this:



Each functionality, such as product catalog, order management, and payments, is managed by its own service, often with its own database.

Pros

- Scalability: Scale individual services independently based on load.
- Resilience: A failure in one service doesn't necessarily bring down the entire application.
- Flexibility: Use different technologies or programming languages for different services.
- Faster Development: Teams can work on different services simultaneously without affecting others.
- Continuous Deployment: Deploy changes to individual services without affecting the rest of the system.

Cons

- Complexity: Managing multiple services introduces operational overhead.
- Inter-Service Communication: Requires robust mechanisms to handle communication failures and latency.
- Data Consistency: Achieving consistency across services can be challenging. Transaction across services is difficult.
- Higher Initial Cost: Requires more infrastructure and tools for orchestration and monitoring.

How to Answer System Design Interview Questions

Most system design interview questions expect candidates to lean towards microservices architecture, especially for large-scale systems. The question then is primarily about how to decompose the problem's requirements into services.

Real-World Advice

While microservices are often the go-to solution in interviews, they are not always the best choice in real-world scenarios. For small-scale applications, a monolithic architecture can be perfectly sufficient. Here's why:

- Premature Optimization: Building a complex system of microservices without fully understanding the business domain can lead to wasted effort and resources.
- Iterative Learning: A monolith allows you to learn and refine your domain model before introducing the overhead of microservices.

As Martin Fowler wisely noted, "Almost all the cases where I've heard of a system that was built as a microservice system from scratch, it has ended in serious trouble... you shouldn't start a new project with microservices, even if you're sure your application will be big enough to make it worthwhile." The best time to refactor into microservices is when you have outgrown your monolithic architecture and have a deep understanding of your system and its requirements.